

Modern CSS Layout

1- Master CSS Grid fundamentals

2- Create complex 2D layouts easily

3- Use grid template areas

4- Combine Grid with Flexbox

Master two-dimensional layouts!

CSS GRID

SCROLL

Quick Recap

Last Week: Flexbox Fundamentals

FLEXBOX CONTAINER Properties:

- display: flex
- flex-direction (row, column)
- justify-content (space-between, center, etc.)
- align-items (flex-start, center, flex-end)
- flex-wrap
- gap

FLEXBOX ITEM Properties:

- flex-grow
- flex-shrink
- flex-basis
- flex (shorthand)



What You Built:

- Responsive navigation bars
- Centered content layouts
- Card rows with equal spacing

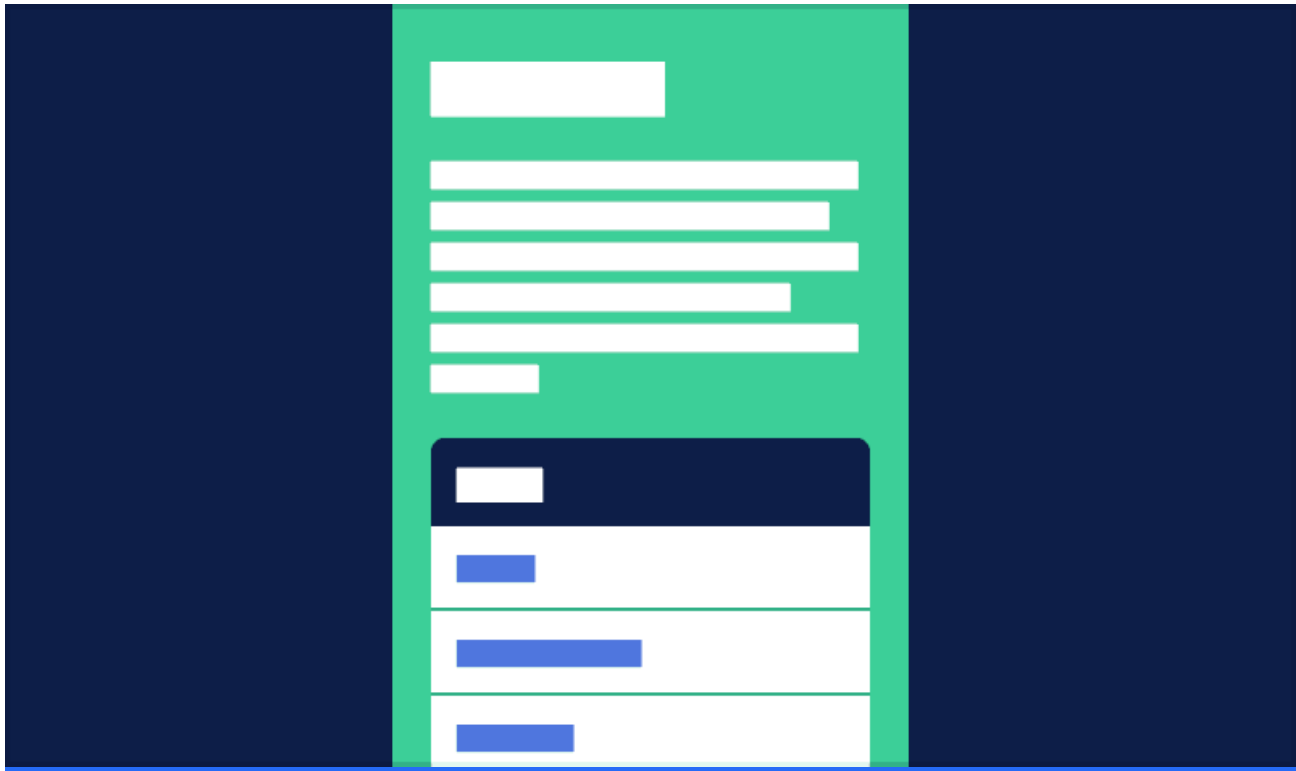
One-dimensional layouts = Flexbox

But what about TWO dimensions?

Today's Goals

By the end of this reading, you will:

- Understand Grid fundamentals (rows + columns)
- Create photo galleries like Instagram
- Use grid-template-areas (visual layout code!)
- Combine Grid + Flexbox together
- Build professional page layouts
- Have a Grid cheat sheet for reference



The most powerful layout system in CSS!

Flexbox vs Grid

Two Layout Systems - Both Essential!

FLEXBOX = One Dimension → → → → →

- Items flow in ONE direction (row OR column)
- Example: Navigation bars, button groups

Perfect for:

- Navigation bars
- Button groups
- Toolbars
- Card rows
- Single-line content

CSS GRID = Two Dimensions

- Items arranged in ROWS AND COLUMNS
- Example: Photo galleries, page layouts

Perfect for:

- Photo galleries
- Page layouts
- Dashboards
- Form grids
- Complex structures

Key Difference:

- Flexbox controls one axis at a time
- Grid controls both axes simultaneously

WORK TOGETHER:

- Grid: Page structure
- Flex: Nav bar inside
- Flex: Card content
- Grid: Photo gallery

GRID FUNDAMENTALS



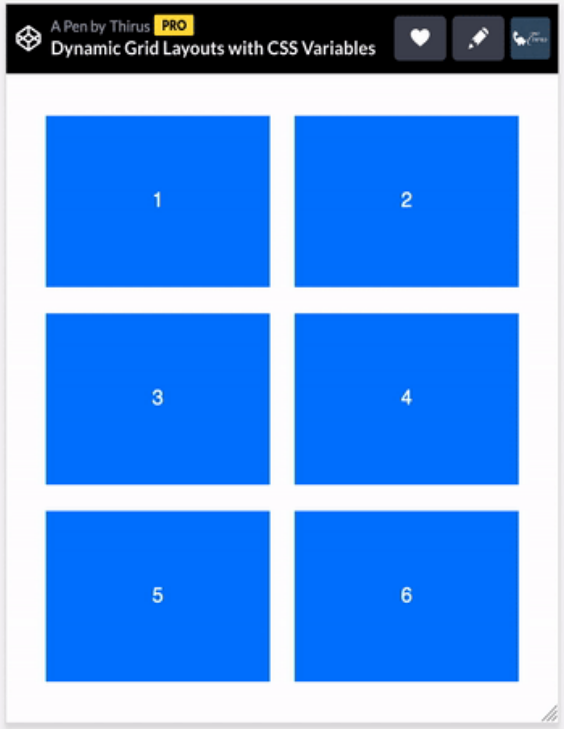
What is CSS Grid?

The Facts:

- Released: 2017 (fully supported in all modern browsers!)
- Browser Support: Chrome, Firefox, Safari, Edge ✓
- Purpose: Complex 2D layouts with clean, simple code
- Responsive: Mobile-friendly by default
- Flexibility: Works alongside Flexbox
- Industry Standard: Used by Apple, Google, Microsoft, Instagram, Pinterest

Why It Changed Everything:

- Before Grid: 50+ lines of hacky float code
- With Grid: 5 lines of clean, readable code



What took years to master, you'll learn in weeks!

Grid Terminology

Essential Grid Vocabulary

- **Grid Container =** Parent element with display: grid
- **Grid Items =** Direct children of the container
- **Grid Lines =** Dividing lines (numbered, not cells!)
- **Grid Tracks =** Rows or columns (space between lines)
- **Grid Cells =** Single square units
- **Grid Areas =** Multiple cells grouped together

Think of it like a spreadsheet:

- Container = The whole spreadsheet
- Items = Content in each cell
- Lines = The borders
- Tracks = Rows/columns
- Cells = Individual squares
- Areas = Selected/merged cells



Don't worry - this looks complex but makes perfect sense visually!

GRID LINES CHEAT SHEET

THE FENCE POST RULE:

- 3 sections → 4 posts
- 3 columns → 4 lines
- n columns → n+1 lines

LINE NUMBERING:

- Start at 1 (not 0!)
- Left to right: 1, 2, 3, 4...
- Top to bottom: 1, 2, 3, 4...
- Negative from end: -1, -2, -3...

POSITIVE NUMBERS:

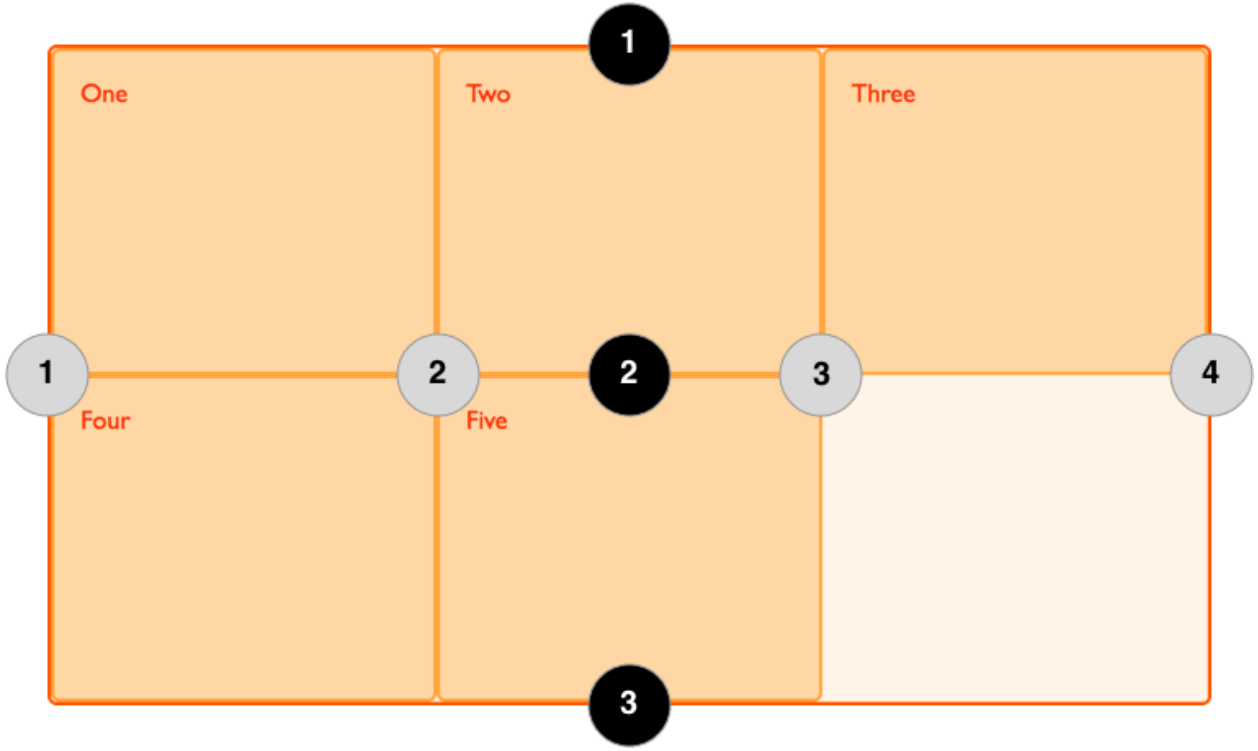
- Line 1 → First line (left/top)
- Line 2 → Second line
- Line 3 → Third line
- Line 4 → Last line (right/bottom)

NEGATIVE NUMBERS:

- Line -4 → First line (same as 1)
- Line -3 → Second line
- Line -2 → Third line
- Line -1 → Last line (same as 4)

Grid Lines

Grid Lines = The Boundaries



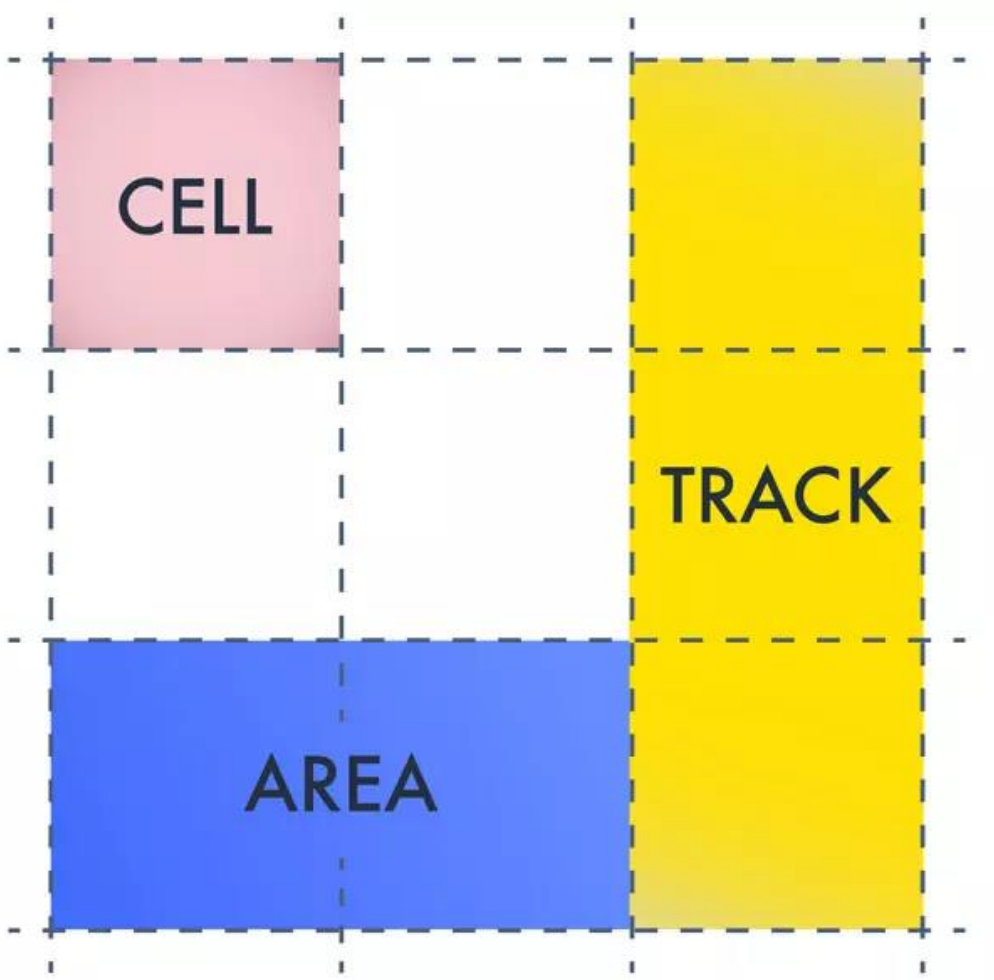
KEY POINTS:

- Lines are numbered starting from 1 (not 0!)
- 3 columns = 4 vertical lines
- 2 rows = 3 horizontal lines
- You position items using line numbers
- Negative numbers count from the end (-1 = last line)

Grid lines are the invisible boundaries that create your grid structure

Grid Tracks

Grid Tracks = Rows & Columns



KEY CONCEPTS:

- Tracks = The space BETWEEN lines
- Column tracks run vertically (top to bottom)
- Row tracks run horizontally (left to right)
- You define track sizes in CSS
- Tracks can be fixed (px) or flexible (fr)

Tracks are like lanes on a highway - they define the flow!

THE FR UNIT



Beyond Pixels - The FR Unit

```css
.grid-container {
  display: grid;
  /* 3 equal columns */
  grid-template-columns: 1fr 1fr 1fr;
  gap: 20px;
}
```

FR = The secret weapon of CSS Grid!

What is FR?

- FR = "Fraction" unit
- Divides available space proportionally
- Accounts for gaps automatically
- Responsive by default

Why FR is Better than %:

- Percentages: Complex math with gaps
- FR: Grid does the math for you!

FR Unit Example

```css
.container-grid {
    display: grid;
    /* Example 1: Equal Columns */
    grid-template-columns: 1fr 1fr 1fr;
    /* Result: All three columns equal width (33.33% each) */

    /* Example 2: Weighted Columns */
    grid-template-columns: 2fr 1fr 1fr;
    /* Result: First column twice as wide (50%, 25%, 25%) */

    /* Example 3: Fixed + Flexible */
    grid-template-columns: 200px 1fr 1fr;
    /* Result: First column always 200px, others split remaining space */

    /* Example 4: Sidebar Layout */
    grid-template-columns: 300px 1fr;
}
```

FR = The secret weapon of CSS Grid!

The repeat() Function

```
/* The Problem: Repetitive and hard to maintain */
grid-template-columns: 1fr 1fr 1fr 1fr 1fr 1fr;
/* What if you want 12 columns? 😰 */

/* The Solution: Clean and easy to change */
grid-template-columns: repeat(6, 1fr);
/* Syntax: repeat(count, size) */
```

More Examples:

```
/* 4 equal columns */
grid-template-columns: repeat(4, 1fr);

/* 5 columns at 200px each */
grid-template-columns: repeat(5, 200px);

/* 3 columns with complex pattern */
grid-template-columns: repeat(3, 1fr 2fr);
/* Results in: 1fr 2fr 1fr 2fr 1fr 2fr */
```

repeat() = Don't Repeat Yourself!

Cleaner code = fewer bugs!

The minmax() Function

minmax() = Flexible with Limits

```css
/* The Power of Boundaries: */
.grid-container {
  grid-template-columns: repeat(3, minmax(200px, 1fr));
}
```

What This Means:

- Minimum: Never smaller than 200px
- Maximum: Can grow to 1fr (fill available space)
- "At least 200px, but grow if there's room"

Real-World Examples:

```css
/* Cards that don't get too small or too big */
grid-template-columns: repeat(4, minmax(250px, 400px));

/* Sidebar with limits */
grid-template-columns: minmax(200px, 300px) 1fr;

/* Content area with readable width */
grid-template-columns: 1fr minmax(300px, 800px) 1fr;
```

Perfect for responsive designs without media queries!

auto-fit vs auto-fill

auto-fit & auto-fill = Magic Responsive Grids!

```css
.grid-container {
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
}
```

- Creates as many columns as fit the space!
- Fully responsive - NO media queries needed!

Real-World Examples:

```css
/* Items stretch to fill space */
grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));

/* Items stay same size, empty space remains */
grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));
```

auto-fit vs auto-fill:

auto-fit:

- Collapses empty tracks
- Stretches items to fill available space
- Best for: Unknown number of items

auto-fill:

- Preserves empty tracks
- Items maintain their size
- Best for: Fixed item sizes

99% of the time, you want auto-fit!

PLACING ITEMS



Placing Items with Grid Lines

Default: Items auto-place in order
But you can control EXACTLY where items go!

Using Line Numbers:

```
.item {
  grid-column-start: 1; /* Start at vertical line 1 */
  grid-column-end: 3;   /* End at vertical line 3 (spans 2 columns) */
  grid-row-start: 1;    /* Start at horizontal line 1 */
  grid-row-end: 2;      /* End at horizontal line 2 (spans 1 row) */
}
```

Shorthand (Better!):

```
.item {
  grid-column: 1 / 3;  /* start / end */
  grid-row: 1 / 2;
}
```

Think: "From line X to line Y"

Placing Items Using span

The span Keyword - Easier Than Line Numbers!
Instead of calculating line numbers, just say how many tracks to span:

```
/* Line Numbers Method: */
.item {
  grid-column: 2 / 4;  /* Math: start at 2, end at 4 = spans 2 */
  grid-row: 1 / 4;     /* Math: start at 1, end at 4 = spans 3 */
}

/* Span Method (Simpler!): */
.item {
  grid-column: span 2;  /* Span 2 columns from wherever you are */
  grid-row: span 3;     /* Span 3 rows from wherever you are */
}

/* You Can Mix Both: */
.item {
  grid-column: 2 / span 3;  /* Start at line 2, span 3 columns */
  grid-row: span 2;         /* Span 2 rows from auto position */
}
```

From wherever it is, span this many tracks

TEMPLATE AREAS



The Most Elegant Feature - Grid Template Areas

What if you could DRAW your layout directly in CSS?

Traditional Grid Placement:

```css
/* Traditional Grid Placement: */
.header { grid-column: 1 / 4; grid-row: 1 / 2; }
.sidebar { grid-column: 1 / 2; grid-row: 2 / 4; }
.content { grid-column: 2 / 4; grid-row: 2 / 3; }
/* ...complex and hard to visualize */
```

Grid Template Areas:

```css
/* Grid Template Areas: */
.page-layout {
    display: grid;
    grid-template-columns: 200px 1fr;
    grid-template-rows: 80px 1fr 60px;
    grid-template-areas:
        "header  header  header"
        "sidebar content content"
        "sidebar footer  footer";
}
```

You can literally SEE the layout in your code!

Named Areas Example

How Grid Template Areas Work

Step 1: Define the Grid and Name the Areas

```css
/* Grid Template Areas: */
.page-layout {
  display: grid;
  grid-template-columns: 200px 1fr;
  grid-template-rows: 80px 1fr 60px;
  grid-template-areas:
    "header header"   /* Row 1: header spans both columns */
    "sidebar content" /* Row 2: sidebar left, content right */
    "footer footer";  /* Row 3: footer spans both columns */
  gap: 20px;
}
```

Step 2: Assign Items to Named Areas

```css
.header  { grid-area: header; }
.sidebar { grid-area: sidebar; }
.content { grid-area: content; }
.footer  { grid-area: footer; }
```

Rules:

- Each string = one row
- Each word = one column
- Same name = item spans those cells
- Names must form rectangles (no L-shapes!)

WHY USE repeat() FUNCTION?



Advanced Gallery – Varied Sizes

Results in Pinterest-Style Layout:

- Most items: Standard size (1 column × 1 row)
- Every 3rd item: Wide (2 columns × 1 row)
- Every 5th item: Tall (1 column × 2 rows)
- Some items: Both! (2 columns × 2 rows when 3n AND 5n)

Creates visual interest and variety!
Like Instagram Explore or Pinterest!

```css
.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  grid-auto-rows: 250px;
  gap: 16px;
}

.gallery-item img {
    width: 100%;
    height: 100%;
    object-fit: cover;
    display: block;
}

/* Every 3rd item spans 2 columns */
.gallery-item:nth-child(3n) {
  grid-column: span 2;
}

/* Every 5th item spans 2 rows */
.gallery-item:nth-child(5n) {
  grid-row: span 2;
}
```

Key Takeaways

1- One Line Magic

`repeat(auto-fit, minmax(250px, 1fr))` creates a fully responsive gallery with ONE line of CSS!

2- No Media Queries

The browser automatically calculates how many columns fit. Truly responsive without breakpoints!

3- Simple HTML

Just a container with repeating items. Grid handles all the layout complexity!

4- Adjust Minimum Size

Change `250px` to any size that fits your content. That's the only value you need to tweak!

5- Universal Pattern

Use this for product grids, portfolios, image galleries, card layouts - everywhere!

6- You'll Use This Daily

This is THE most practical Grid pattern. Every developer should know this by heart!

COMBINING GRID & FLEXBOX



Grid + Flexbox = Perfection

They're Not Competitors - They're Teammates!

Use GRID for:

- Overall page structure
- Major layout sections
- 2D positioning (rows AND columns)
- Page-level organization

Use FLEXBOX for:

- Components inside Grid areas
- Navigation bars
- Button groups
- Card internals
- Centering content within grid items

The Pattern: Grid creates the structure → Flexbox styles the components inside

Real Example:

- Grid: Defines header, sidebar, content, footer layout
- Flexbox: Centers logo and menu inside header
- Flexbox: Arranges buttons inside content area
- Flexbox: Aligns footer text and links

Example - Grid + Flexbox

What's Happening:

- Grid creates page structure (header, sidebar, content)
- Flexbox arranges items INSIDE header (logo ← → menu)
- Flexbox stacks content vertically
- Flexbox arranges buttons horizontally

```css
/* Grid for overall layout */
.page {
  display: grid;
  grid-template-columns: 200px 1fr;
  grid-template-areas:
    "header header"
    "sidebar content";
  gap: 20px;
}

/* Flexbox inside header (Grid item) */
.header {
  grid-area: header;
  display: flex; /* Flexbox! */
  justify-content: space-between;
  align-items: center;
  padding: 20px;
}

/* Flexbox inside content area (Grid item) */
.content {
  grid-area: content;
  display: flex; /* Flexbox! */
  flex-direction: column;
  gap: 20px;
}

/* Flexbox for button group inside content */
.button-group {
  display: flex; /* Flexbox! */
  gap: 10px;
  justify-content: center;
}
```

Key Takeaways

1- Not Competitors

Grid and Flexbox are NOT competing technologies. They're designed to work together!

2- Grid for Structure

Use Grid for overall page structure and major layout sections. It's perfect for 2D layouts.

3- Flexbox for Components

Use Flexbox inside Grid areas for component layout. Perfect for nav bars, buttons, card content.

4- Nesting is Normal

Nest Flexbox inside Grid all the time! This is the professional, standard approach.

5- Real Projects Use Both

Every real website uses both Grid and Flexbox together. This is how professionals build layouts!

6- Best Practice

Start with Grid for page structure, then use Flexbox for components inside. Clean and maintainable!

GRID ALIGNMENT



Aligning Items in Grid

Control how items sit inside their grid cells!

For the Container (align ALL items):

- **justify-items** - Align items horizontally within cells Values: start | end | center | stretch
- **align-items** - Align items vertically within cells Values: start | end | center | stretch
- **justify-content** - Align entire grid horizontally Values: start | end | center | stretch | space-between | space-around
- **align-content** - Align entire grid vertically Values: start | end | center | stretch | space-between | space-around

Similar to Flexbox!

If you understood Flexbox alignment, this is familiar!

Key Difference:

- justify = horizontal (inline axis)
- align = vertical (block axis)

justify-items & align-items

Common Values:

- start - Items align to start of cell
- end - Items align to end of cell
- center - Items centered in cell (perfect centering!)
- stretch - Items fill entire cell (DEFAULT)

```
.grid-container {
  display: grid;
  grid-template-columns: repeat(3, 200px);
  grid-template-rows: repeat(3, 150px);
  gap: 10px;
  justify-items: center;
  align-items: center;

  /* Horizontal alignment of items within cells
    justify-items: start | end | center | stretch;
  */

  /* Vertical alignment of items within cells
    align-items: start | end | center | stretch;
  */
}
```

Individual Item Alignment

Control how ONE specific item aligns (overrides container setting):

```css
.grid-container {
  display: grid;
  justify-items: start; /* All items align left */
  align-items: start;   /* All items align top */
}

.special-item {
  justify-self: center; /* THIS item only - centered horizontally */
  align-self: end;      /* THIS item only - aligned to bottom */
}
```

Properties:

- justify-self - Align THIS item horizontally within its cell Values: start | end | center | stretch
- align-self - Align THIS item vertically within its cell Values: start | end | center | stretch

Key Takeaways

1- Similar to Flexbox

If you know Flexbox alignment, Grid alignment uses the same properties and values!

2- Default is stretch

By default, items stretch to fill their cells. Use `center` for perfect centering!

3- Items vs Self

`*-items` affects ALL items. `*-self` affects ONE specific item.

4- Justify = Horizontal

`justify-*` controls horizontal alignment (inline axis).

5- Align = Vertical

`align-*` controls vertical alignment (block axis).

6- Perfect Centering

`justify-items: center; align-items: center;` gives perfect centering!


Placing items with grid lines?
Building image galleries?
CSS GRID Fundamental?
The first PROJECT?
Combing Grid + Flexbox?
Anything unclear?
HAVE ANY
QUESTIONS?